



Bài 4 Cấu trúc dữ liệu và giải thuật 2

Công nghệ thông tin (Đại học Xây dựng Hà Nội)



Scan to open on Studocu

3.8. Cây tìm kiếm nhị phân BST – Bynary Search Tree

3.8.1. *Khái niệm về Cây nhị phân tìm kiếm*

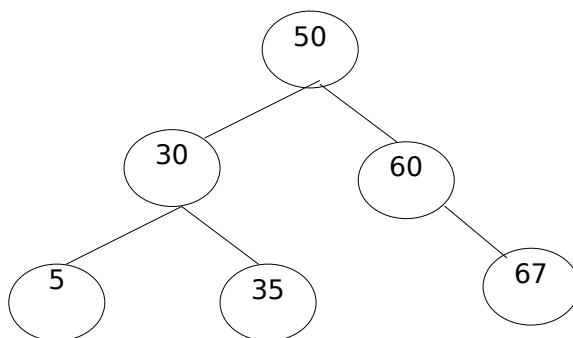
Trong một cây nhị phân mà giá trị ở mỗi nút cha lớn hơn giá trị ở nút con bên trái của nó (nếu có) và nhỏ hơn giá trị ở nút con bên phải của nó thì gọi là cây tìm kiếm nhị phân.

Sở dĩ gọi là cây tìm kiếm nhị phân (Binary Search Tree- BST) vì có thể dùng thuật toán gần giống tìm kiếm nhị phân để tìm kiếm trên nó.

Dữ liệu ở mỗi nút (còn được gọi là khóa của đỉnh, tìm theo khóa – key) có các thuộc tính sau:

- ❖ Dữ liệu ở các nút phải là một kiểu dữ liệu có thể so sánh được.
- ❖ Dữ liệu ở các nút không được giống nhau, nó là một khóa để phân biệt với các nút khác.
- ❖ Với mỗi nút cha, dữ liệu của tất cả các nút trong cây con bên trái nhỏ hơn dữ liệu của nút đó.
- ❖ Với mỗi nút cha, dữ liệu của tất cả các nút trong cây con bên phải lớn hơn dữ liệu của nút đó.

Ví dụ, cho cây nhị phân tìm kiếm :



3.8.2. *Cài đặt cây nhị phân tìm kiếm BST*

Về nguyên tắc, cài đặt cây nhị phân tìm kiếm cũng giống như cài đặt cây nhị phân.

*Khai báo cấu trúc nút cho cây **BST***

```
struct BSTNode
{
    int du_lieu;
    BSTNode* Lchild; //Lchild là con trỏ trỏ sang trái
    BSTNode* Rchild; //Rchild là con trỏ trỏ sang phải
};
```

3.8.3. *Tạo node cho BST*

```
BSTNode* TaoNode(int du_lieu)
{
    BSTNode* p = new BSTNode ; //khai báo con trỏ p có kiểu BSTNode = cấp bộ nhớ
    //cho *p theo kiểu cấu trúc BSTNode
    p->du_lieu = du_lieu;
    p->Lchild = p->Rchild = NULL; //các con trỏ trái,phải chưa trỏ đi đâu cả
    return p;
}
```

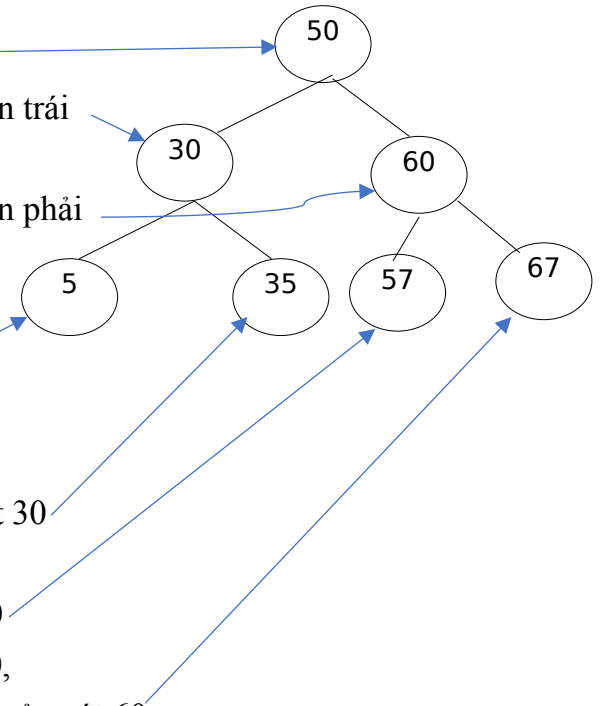
3.8.4. Tạo cây BST

Để tạo cây, ta cần tạo một nút để nhận dữ liệu bằng hàm TaoNode(int du_lieu), sau đó tìm vị trí nút cần chèn và chèn nút.

Ví dụ tạo cây nhị phân tìm kiếm với dữ liệu đã cho : 50, 30, 60, 5, 35, 57, 67

Trình tự thực hiện :

- Nhập nút gốc 50
- Nhập 30, kiểm tra $30 < 50$ nên chèn sang cây con bên trái
-
- Nhập 60, kiểm tra $60 > 50$ nên chèn sang cây con bên phải
- Nhập 5, kiểm tra $5 < 50$ – chuyển sang trái
- Gặp nút 30, so sánh $5 < 30$ – chuyển sang trái
- Chèn 5 sang cây con trái của nút 30
- Nhập 35, kiểm tra $35 < 50$ – chuyển sang trái
- Gặp nút 30, so sánh $35 > 30$ – chuyển sang phải
- Chèn 35 sang cây con phải của nút 30
- Nhập 57, kiểm tra $50 < 57$ – sang cây con phải, gặp $60 > 57$ nên chèn 57 sang trái của 60
- Nhập 67, $67 > 50$ nên chuyển sang phải; gặp nút 60, so sánh $60 < 67$ -chèn 67 sang cây con phải của nút 60



Bài tập : dựa theo cách chèn nút trên đây hãy vẽ sơ đồ khối thuật toán tìm vị trí để chèn nút vào cây BST.

3.8.4.1. Tìm vị trí để chèn thêm nút vào bên trái hay bên phải của nút đã cho.

Bài toán : cho một cây nhị phân tìm kiếm BST và một dữ liệu, cần tìm vị trí để chèn nút nhận giá trị dữ liệu đó.

Ý tưởng thuật toán : bắt đầu từ gốc.

Nếu gốc == NULL , cây rỗng, return NULL;

Nếu gốc != NULL thì tạo nút p=gốc, tạo nút vi_tri để lưu vị trí cần tìm, lúc đầu gán vi_tri=p;

Thực hiện vòng lặp while(p!=NULL) :

```
{ vi_tri = p;
  nếu (p->du_lieu > du_lieu) thì { p = p->Lchild; } nếu không thì {p = p->Rchild; }
```

kết quả, cho vi_tri cần tìm để chèn du_lieu vào cây BST.

Ta có hàm :

BSTNode* TimVetriChen(BSTNode* GOC, int du_lieu)// Tìm vị trí để chèn nút

```
{ if (GOC == NULL)
  { return NULL; }
  BSTNode* p = GOC;
  BSTNode* f = p;
  while (p != NULL)
  { f = p;
    if (p->du_lieu > du_lieu)
```

```

        { p = p->Lchild; }
    else
        {p = p->Rchild; }
    }
    return f;
}

```

3.8.4.2.Chèn nút

Sau khi xác định được vị trí chèn nút, sẽ tiến hành chèn nút.

Ý tưởng thuật toán :

- Gọi hàm TaoNode(du_lieu) để tạo nút p với dữ liệu mới
- Nếu gốc=NULL thì gốc nhận nút p
- Nếu gốc !=NULL thì :
 - + Gọi hàm TimVitriChen(GOC, du_lieu) để cho vị trí chèn
 - + Nếu vi_tri !=NULL thì kiểm tra tiếp :
 - Nếu vi_tri->du_lieu > du_lieu thì chèn p vào bên trái nút vi_tri
 - Nếu vi_tri->du_lieu < du_lieu thì chèn p vào bên phải nút vi_tri

Ta có hàm :

```

void ChenNode(BSTNode* &GOC, int du_lieu) // Chèn node
{ BSTNode* p = TaoNode(du_lieu); //TaoNode() return p để gán cho *p được tạo ở bên trái
  if (GOC == NULL)
  { GOC = p ;
    return;
  }
  else
  { BSTNode* vi_tri = TimVitriChen(GOC, du_lieu); // tìm vị trí vi_tri để chèn nút p vào
    trái||phải
    if (vi_tri != NULL)
    { if (vi_tri->du_lieu > du_lieu)
      {vi_tri->Lchild = p;} // chèn p vào bên trái vi_tri
      else
      { vi_tri->Rchild = p; } //chèn p vào bên phải vi_tri
    }
  }
}

```

Như vậy để thêm nút vào cây cần 3 hàm :

- Tạo nút p với dữ liệu mới,
- Tìm vi_tri để đưa nút p vào,
- Chèn nút p vào trái||phải của vi_tri.

3.8.4.3.Tạo cây

Chú ý :

Trong hàm tạo cây dùng hàm ChenNode(), trong hàm ChenNode() phải gọi đến 2 hàm TaoNode(du_lieu) và TimVitriChen(GOC, du_lieu) .

```
void TaoCay(BSTNode* &GOC, int a[], int n)
{for(int i = 0; i < n; i++)
    {ChenNode(GOC, a[i]);}
}
```

3.8.5.Đếm số nút trong cây

Đếm số nút trong cây phải đi từ gốc, rồi di chuyển tới cây con bên trái, cây con bên phải cho đến các nút lá cuối cùng. Vì vậy cần đến hàm chuyển đến nút con trái, chuyển đến nút con phải.

a) Chuyển hướng lên cây con trái

```
BSTNode* Lchild(BSTNode* GOC)
{ if(GOC!=NULL) return GOC->Lchild;
  else return NULL;
}
```

b)Chuyển hướng đến cây con phải

```
BSTNode* Rchild(BSTNode* GOC)
{ if(GOC!=NULL) return GOC->Rchild;
  else return NULL;
}
```

c)Hàm đếm số nút

Để đếm số nút trong cây, sẽ đi từ gốc và gọi 2 hàm định hướng lên cây con trái Lchild(BSTNode* GOC) và định hướng lên cây con phải Rchild(BSTNode* GOC), theo kiểu đệ quy tiến tiếp đến từng nút, mỗi nút lại đếm tiếp theo hướng nút con trái, nút con phải,..., cho đến các lá.

Ta có hàm :

```
int DemNode(BSTNode* GOC) //Đếm số nút trong cây
{ if(GOC==NULL) return 0;
  else return (1+DemNode(Lchild(GOC))+ DemNode(Rchild(GOC)));
}
```

3.8.6.Duyệt cây BST

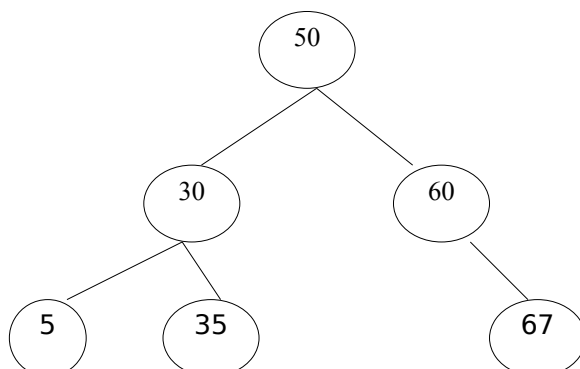
Duyệt cây BST cũng giống như duyệt cây nhị phân thường đã xét ở trên,cũng theo 3 kiểu :

- InOrder – duyệt theo thứ tự dữ liệu giữa
- PreOrder - duyệt theo thứ tự dữ liệu trước
- PostOrder - duyệt theo thứ tự dữ liệu sau

3.8.7. Tìm kiếm nút trong cây BST

Việc tìm kiếm nút trong cây BST khác với cây nhị phân thường do thuộc tính

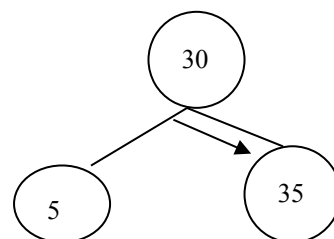
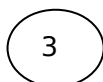
- ❖ Dữ liệu của tất cả các nút trong cây con bên trái nhỏ hơn dữ liệu của nút cha.
- ❖ Dữ liệu của tất cả các nút trong cây con bên phải lớn hơn dữ liệu của nút cha.



Giả sử muốn tìm nút 35. Ta bắt đầu so sánh giá trị của nút cần tìm từ nút gốc, vì $35 < 50 \rightarrow$ Giá trị cần tìm ở bên trái nút gốc $\rightarrow$ nghĩa là nó cây con phía trái

So sánh tiếp 35 với gốc cây con: vì $35 > 30 \rightarrow$ ở phía cây con bên phải

Nó là duy nhất. $\rightarrow$ kết thúc.



Thuật toán tìm nút như sau:

- Nếu nút hiện thời có giá trị = giá trị cần tìm, trả về true và **kết thúc**.
- Nếu nút hiện thời có giá trị $>$ giá trị cần tìm, gọi đệ quy tìm ở cây con bên trái.
- Nếu nút hiện thời có giá trị $<$ giá trị cần tìm, gọi đệ quy tìm ở cây con bên phải
- Nếu tìm đến hết cây (nút đó = NULL) mà không tìm thấy, trả về false, thông báo “không tìm thấy” và **kết thúc**.

Bài tập : vẽ sơ đồ khối thuật toán tìm nút trong cây BST theo giá trị dữ liệu của nút

Chú ý : cần phải vẽ ra sơ đồ thuật toán, hiểu được cách giải bài toán thì mới viết được các dòng lệnh.

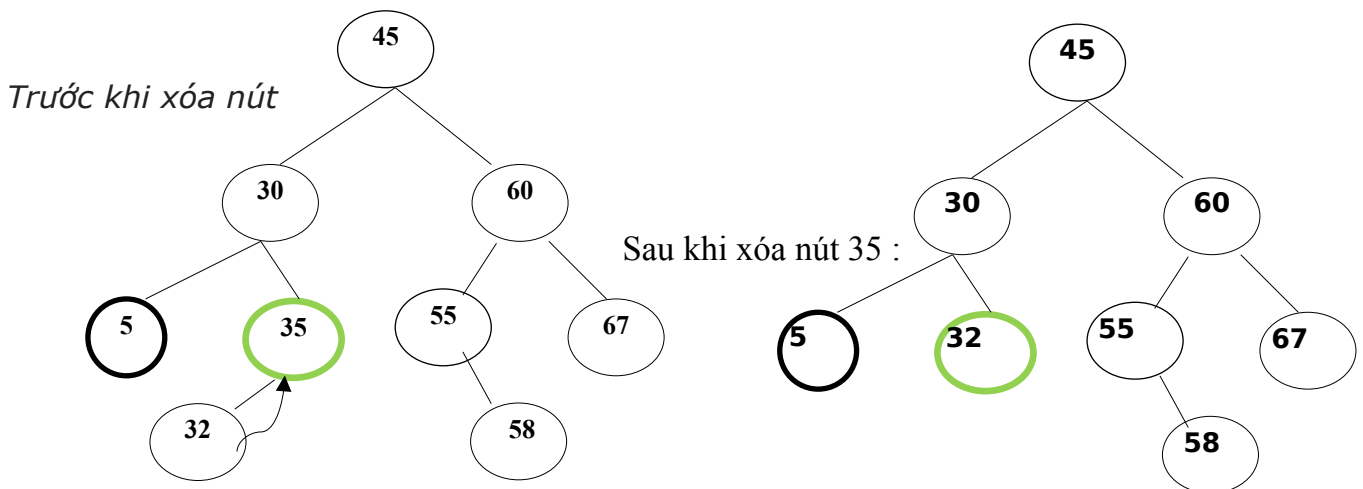
```
BSTNode* TimNode(BSTNode* GOC, int du_lieu)//tìm theo dữ liệu đã cho
{
    if(GOC == NULL)
    {
        return NULL;
    }
    if (GOC->du_lieu == du_lieu)
    {
        cout<<" Da tìm thay nut "<<GOC->du_lieu<<"\n";
        return GOC;
    }
    if (GOC->du_lieu > du_lieu)
    {
        TimNode(GOC->Lchild, du_lieu);
    }
    else
    {
        TimNode(GOC->Rchild, du_lieu);
    }
}
```

3.8.8.Xóa nút

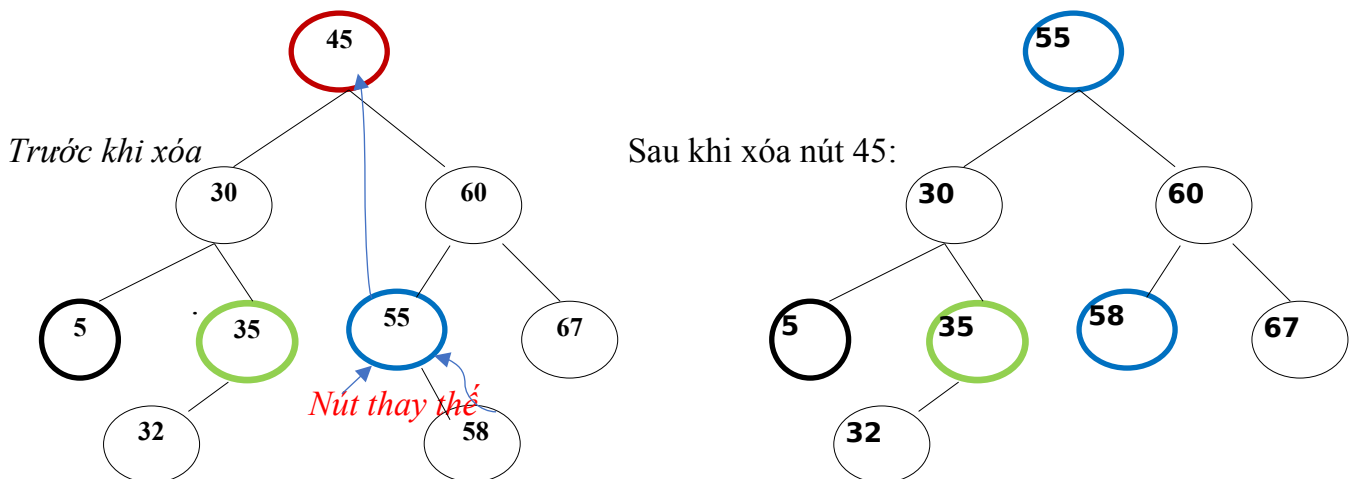
Để xóa một nút, ta tìm đến nút đó, rồi xét :

- Nếu nó là nút lá thì xóa nó
- Nếu nó có 1 nút con thì thay nó bằng nút con của nó rồi xóa nút con
- Nếu nút cần xóa có 2 nút con thì :
 - + Tìm nút có giá trị lớn hơn mọi nút bên trái và nhỏ hơn mọi nút con bên phải của nút cần xóa, nút này sẽ là nút của cây con trái nhất của cây con bên phải của nút cần xóa, đặt nó là nút *Nút thay thế*
 - + Thay thế nút cần xóa bằng *Nút thay thế*
 - + Gọi đệ quy Xóa nút *Nút thay thế*

Ví dụ. xóa nút 35 ở cây dưới đây : đưa nút 32 về nút 35 rồi xóa nút 32



Ví dụ. xóa nút 45, tìm nút thay thế là 55, đưa 55 về chỗ 45, đưa 58 về 55



Hàm xóa nút

```

BSTNode* XoaNut(BSTNode *GOC, int du_lieu)
{static int dlтт = -1 ;
  if (GOC == NULL) return GOC;
  if (GOC->du_lieu == du_lieu)
  { if (GOC->Lchild == NULL && GOC->Rchild == NULL)
    { delete GOC; return NULL ; }
    if ( (GOC->Lchild == NULL )&&(GOC->Rchild != NULL))
    {BSTNode* newGOC = GOC->Rchild;
      delete GOC; return newGOC; }
    if ( (GOC->Lchild != NULL )&&(GOC->Rchild == NULL))
    { BSTNode* newGOC = GOC->Lchild;
      delete GOC ; return newGOC; }
    if (GOC->Lchild != NULL && GOC->Rchild != NULL)
    { // Tim nút thay thế
      BSTNode *NutThayThe = GOC->Rchild;
      while (NutThayThe->Lchild != NULL) NutThayThe = NutThayThe->Lchild;
      GOC->du_lieu = NutThayThe->du_lieu; dlтт = NutThayThe->du_lieu;
      cout<<" \ Nut Thay The co du lieu = "<<NutThayThe->du_lieu<<" \n ";
      TimXoa(GOC->Rchild, NutThayThe->du_lieu);
      return GOC; }
    if (GOC->Lchild != NULL) return GOC->Lchild;
      else      return GOC->Rchild;
    }
  if (du_lieu > GOC->du_lieu) GOC->Rchild = TimXoa(GOC->Rchild, du_lieu);
    else      GOC->Lchild = TimXoa(GOC->Lchild, du_lieu);
  return GOC;
}

```

Bài tập về nhà : 1) Vẽ sơ đồ khối thuật toán Xóa nút

2) Ghép nối các khai báo, các hàm đã cho và viết thêm hàm main() có các chức năng như các hàm đã học.

Hướng dẫn phân lập trình :

1) Nhập các dòng header

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

2) Khai báo cấu trúc nút

```
struct BSTNode
```

3) Các hàm

- BSTNode* TaoNode(int du_lieu)
- BSTNode* TimNodeChen(BSTNode* GOC, int du_lieu)
- ChenNode(BSTNode* &GOC, int du_lieu)
- TaoCay(BSTNode* &GOC, int a[], int n)
- BSTNode* TimNode(BSTNode* GOC, int du_lieu)
- Duyệt theo InOrder - thu tu giữa
- Duyệt theo thu tu trước PreOrder
- Duyệt theo thu tu sau PostOrder
- Chuyển hướng nút con trái BSTNode* Lchild(BSTNode* GOC)
- Chuyển hướng nút con phải BSTNode* Rchild(BSTNode* GOC)
- Đếm số nút DemNode(BSTNode* GOC) //Đếm số nút trong cây
- Xóa nút BSTNode* XoaNut(BSTNode *GOC, int du_lieu)

4) Hàm main() bắt đầu bằng các dòng :

```
{BSTNode* GOC = NULL;
```

```
int a[] = {.. ,..... };
```

```
int du_lieu;
```

```
int n=sizeof(a)/sizeof(a[0]);
```

```
TaoCay(GOC, a, n);
```

Tiếp đến gọi các hàm :

- ✓ DemNode(GOC)
- ✓ Duyệt cây theo thu tu giữa InOrder
- ✓ Duyệt cây theo thu tu trước PreOrder
- ✓ Duyệt cây theo thu tu sau PostOrder
- ✓ Nhập du_lieu gọi TimNode(GOC,du_lieu);
- ✓ Nhập du_lieu cho nút cần Xóa gọi TimXoa(GOC,du_lieu);
- ✓ Duyệt cây theo thu tu giữa InOrder sau khi xóa nút

....

Tự thêm các lệnh khác

.....

```
system("pause");
```

```
return 0;
```

```
}
```

Ví dụ đã chạy chương trình cho bộ số liệu a[] = { 45,30,5,35,32,60,55,58,67 };

Kết quả chạy chương trình :

```

So nut cua cay hien co : 9

Duyet cay theo thu tu giữa InOrder :
-5-30-32-35-45-55-58-60-67

Duyet cay theo thu tu trước PreOrder :
-45-30-5-35-32-60-55-58-67

Duyet cay theo thu tu sau PostOrder :

```

```
Nhap nut can tim : 35
Da tim thay nut 35
Press any key to continue . . .

Nhap nut can Xoa : 5
Nut 5 da bi xoa

Duyet cay theo thu tu giua InOrder sau khi xoa nut :
-30-32-35-45-55-58-60-67
Press any key to continue . . .

Nhap nut can Xoa : 35
Nut 35 da bi xoa

Duyet cay theo thu tu giua InOrder sau khi xoa nut :
-30-32-45-55-58-60-67
Press any key to continue . . .

Nhap nut can Xoa : 45
Nut Thay The co du lieu = 55
Nut 45 da bi xoa

Duyet cay theo thu tu giua InOrder sau khi xoa nut :
-30-32-55-58-60-67
Press any key to continue . . .

Process returned 0 (0x0)   execution time : 77.954 s
Press any key to continue.
```

Windows (CR+LF) UTF-8

Line 47, Col 17, Pos 905

Insert